

Analysis

This task looks like a direct dynamic optimization problem but this is not correct. There have to be made some important observations and after them the task becomes more straightforward.

The first subtask is for 10 points. No observations are needed to be made here – considering the constraints it is enough for every substring to be viewed and to be counted how many times every type of letter is met. Next the counts for every type of letter are checked and if they are all equal then the current substring is counted in the answer. The expected complexity is $O(n^3 + kn^2)$.

The second subtask is for 20 points. Here the idea is to optimize what we did before. We will remove the inner cycle – we will calculate the count for every type of letter for a current substring in constant complexity. There are different ways to do that. The author precomputes the prefix counts for every type of letter and with this the count for every type of letter for a current substring can be calculated as a subtraction of prefix counts. So the complexity here is $O(kn^2)$.

The third subtask is for 30 points. We have to make the first observation in the task which is for an easier variant – when we only have two types of letters. Let us do the following change of the letters in the string. We replace the first type of letters with the number +1 and the second type – with the number -1 (negative 1). In this way our string becomes an array of numbers. The needed substrings are with equal number of letters from the first and the second type i.e. the sum of the numbers in the array in these positions would be 0 which means we are looking for subintervals with 0 sum in the array. This task is solved easier than the original. Let us calculate the prefix sums in the array *pref* (we have a fictive prefix sum in the beginning which is 0). The sum of a subinterval of the array from *x* to *y* is $pref[y] - pref[x-1]$. We want this to be 0 i.e. for a given *y* the subintervals ending there with zero sum are with *x*-es for which $pref[x-1] = pref[y]$. To solve the current subtask we can save how many prefix sums are there for every possible value of a sum in the array *cnt* and for a current index *ind* the number of subintervals with the needed property ending there are *cnt[ind]*. We add this to the value of the answer and then we need to make the update: *cnt[ind]*++. The complexity of the described procedure is linear – $O(n)$ but the solution for the whole task isn't so here it can be made a solution with complexity $O(n \log_2 n)$.

The fourth subtask is for 40 points. The previous subtask was useful because the idea can be adapted for the whole task. Let us fix two types of letters – *a* and *b* (these are only fictive and the letters can be different). Let us save the number values in the array *nums*₁. Again in the same way we replace the letters of type *a* with +1, we replace the letters of

type b with -1 but for every letter which is from a different type we replace it with the number 0 . Let we take the second type of letters – b and a new type of letters – c and we will save the numbers in a new array $nums_2$. After we make the replacements and save the numbers using the described scheme we take c and another different type of letters (for example d) and we again fill in an array $nums_3$ and we do this while we still have more unused types of letters. A substring that fulfills the requirement in the statement is such that the sums of the numbers in the corresponding subintervals in the arrays $nums_1, nums_2, \dots, nums_{k-1}$ are all zeroes. Now we can approach analogously as in the previous subtask. Here we have a problem – the array cnt which we used now has to have several numbers as a parameter. We can use hashes which isn't expected to be known by the competitors so the author's solution doesn't use this despite providing complexity of $O(kn)$. We can use the data structure *map* from *STL* but we have to predefine operator and there will be a big constant which will make the solution slower. The author approaches in the following way. Let for every array $nums_i$ we calculate the corresponding prefix array $pref_i$. For every position i we can make this type of sets $S_i\{pref_1, pref_2, \dots, pref_n\}$. In reality we search for pairs of sets S_x and S_y (of course $x < y$) for which $S_x \equiv S_y$. Let we sort the sets S_i . Now the equivalent sets are adjacent. Let we view all groups of equivalent sets and their number is p and every group's length is num_i . The number of all substrings which fulfill the requirement from the statement is $\sum_{i=1}^{p-1} num_i(num_i-1)$. This is easy to be calculated and solves the task for 100 points. The complexity of the described procedure is $O(kn \log_2 n)$ – this is the complexity of the sorting of the sets S_i which is the dominating in the solution.

The thing which shouldn't be forgotten is that we have to make the calculations by the modulo in the statement. In reality it turns out that the answer can't be too big (the more types of letters the small it gets) – using a convenient sample the maximal number for

a given n and k is
$$l \frac{n}{k} J(n+1) - \frac{k l \frac{n}{k} J\left(l \frac{n}{k} J+1\right)}{2}$$
 which for the maximal constraints isn't much

bigger than the modulo. The reason there is a modulo is for deception that the answer will be very big and to prevent the competitors from doing cheat solutions. The task is interesting because of the approach that the letters are replaced with numbers which is the big step towards finding a solution with smaller complexity.

Analysis

In order to be able to solve the problem we have to learn to find the exact moment in which two particles collide. Let us try to find the moment in which the i -th x particle and the j -th y particle collide. We can do that quite easily using a binary search on the moment of collision, but that would have a complexity of $O(\log)$. To find the moment of collision in $O(1)$ let us look at the formulas that describe the position of a certain particle in a specific moment. Let us imagine that the x particles are being shot from point 0 rightwards on the number line, and the y particles are being shot from point L leftwards on the number line. At a certain moment t , the position of the two particles is:

$$P x_i = (t - t x_i) \times v x_i$$

$$P y_j = L - (t - t y_j) \times v y_j$$

This formula could give negative values if we evaluate it for invalid times in which the particle has not been shot yet, but that does not matter. We clearly have a collision if

$$P x_i = P y_j. \text{ That is:}$$

$$(t - t x_i) \times v x_i = L - (t - t y_j) \times v y_j$$

$$t \times v x_i - t x_i \times v x_i = L - t \times v y_j + t y_j \times v y_j$$

$$t \times v x_i + t \times v y_j = L + t y_j \times v y_j + t x_i \times v x_i$$

$$t = \frac{L - t y_j \times v y_j + t x_i \times v x_i}{v x_i + v y_j}$$

And hence we get a direct formula for the moment of collision. It is possible for the collision to happen before 0 or after L on the number line, which means that this collision will surely not happen (those particles will collide with some others before colliding with each other).

Solution with complexity $O(N^2 \times \log N)$

Since we can quickly find the moment of collision between two particles, we can find the moment of collision for every pair of particles and order these collisions chronologically. The total amount of collisions is $O(N^2)$, and sorting them takes $O(N^2 \times \log N)$. After this we can iterate over all collisions chronologically and keep which of the particles have already collided. If we reach a pair in which both particles have not collided with anything yet, then we know that they will collide with each other. In this way we can find not only the first K , but all collisions with the same complexity.

Solution with complexity $O(N \times K \times \log)$

We will describe a solution that finds only the first collision. After finding it we can remove the two particles that collided and repeat the whole process K times.

To find the first collision we will use binary search on the moment of the first collision. Let us fix the time t . We want to know whether the first collision happened before or after t . To do this we can find the furthest x particle and the furthest y particle (by 'furthest' we mean the

one that travelled the largest distance, that is, using the formulas above, the x particle with the largest position and the y particle with the smallest one). Let the positions of the two particles, according to the formulas above, be respectively P_x and P_y . Then:

- If $|P_x - P_y| < \varepsilon$ (where ε is a small enough constant) we can assume that t is the moment of the first collision. The particles which collided are the furthest particles..

Else:

- If $P_x < P_y$, then the first collision has not happened yet and we have to look for a larger value of t
- If $P_x > P_y$, then the first collision already happened and we have to look for a smaller value of t

This solution has a complexity of $O(N \times \log)$ for finding the first collision. The bounds of the binary search in which we search for t are from 0 to the time needed for any particle to be shot and travel a distance of L (since at this moment the particle must have collided with some other one).

Since we are doing a binary search on real values and not integers, it is a good idea to set a limit on the amount of iterations the binary search can perform, since choosing a good ε may be tricky. About 40-50 iterations should be ideal (more may trigger time limits and less could lead to precision errors).

Using this procedure K times we get a solution with total complexity of $O(N \times K \times \log)$.

The problem can be solved in $O(N \times K)$, but this was not required.

Analysis

The main thing that the contestants must have noticed in this problem was that **N** has at most six distinct prime divisors. Note that these 6 primes will be not only in the factorization of **N**, but also in the factorization of all of its divisors – the numbers, Elly is writing.

The first thing the contestants should do before getting to any solution is find the (up to) 6 prime factors of **N**. This can be done quite trivially, using the $O(\sqrt{N})$ algorithm. Since **N** was up to 10^{15} , this would require around 32 million operations – a fraction of a second on a processor manufactured in the past 10 years.

Now, let's get to the main part of the problem. As with many other problems where the question is "how many" and the answer can be pretty large (thus, the modulus), here as well the solution is based on dynamic programming. We'll need to have a state, which tells us which divisors of **N** we can write at any given time.

In order two numbers in the list to be coprime (to have no common divisors greater than or equal to 2), they must share no prime number in their factorization. Thus, keeping track which of the primes have not been used, which have been used once, and which have been used twice should do the trick, right?

Well, almost. Unfortunately, it is not entirely enough and will most likely be a common problem the competitors face. It is fairly intuitive to go this way (use a ternary mask for the state), however we'll show that this is actually incorrect. Luckily for them, the examples are quite strong and reveal this!

The problem with the solution above is that it matters whether two primes (which were used once) were used in the same number or in two different numbers. Consider the following sequences:

1. (2, 3, 30)
2. (6, 30)

The first one is invalid, because 30 has common divisors with both 2 and 3 (two different numbers from the list). However, the second one is valid – 30 still has common divisors with 2 and 3, but this time they are used in the same number 6.

So, we need a state, which both:

- Keeps track which primes were used 0, 1, and 2 times.
- Keeps track which primes were used together.

We have several possible solutions. Let's start with the easiest one.

Approach 1 (keep all occurrences of each factor)

To get to this approach we need to make the observation that the list Elly writes can have at most 12 elements. This should be fairly obvious, as each prime can be used at most twice (and $6 * 2 = 12$).

Instead of storing 0 (not used), 1 (used once), or 2 (used twice) for each prime, we can store not used (no restrictions), used twice (cannot use) or the position, at which it was used. Knowing the position where each of the primes was used helps us solve the problem we outlined above.

We will need 14 values for each prime: 0-11 will represent the position at which it was used, 12 will mean it was not used and 13 will mean it was already used twice. This means a DP with $14^6 = 7,529,536$ states, which is okay from memory point of view (the answer fits into int, thus we'll need around 30 megabytes for the DP table).

In terms of time, for each state we have to choose which divisor of **N** to use. Since we have up to 6 primes, the number of divisors is $2^6 = 64$. This looks bad at first, since, if we do the math, we'll need around $14^6 * 2^6 = 481,890,304$ operations in the worst case – more than we can afford for the given time limit. However, we can easily see that many of the states are invalid (can never be reached), thus shouldn't be counted towards that (see below for explanation why). In fact, running the last example (which has six prime divisors, i.e., should reach as many states as possible) reaches only 170,000 out of the 7,500,000 possible states. With that many actual states, the 64 operations per state are quite okay – this solution runs quite quickly on any input with these constraints, thus it gets 100 points.

Why are the valid states so few?

How can we deduce that many of the states will be invalid? Let's assume that as the first number in the list we use a divisor, which includes multiple factors. This drastically reduces the maximal length of the list. For example, if the first element is **N** itself (thus, uses all 6 factors), the length of the list is suddenly limited to 7.

Moreover, an average divisor of **N** has $(1 + 6) / 2 = 3.5$ (thus 3 or 4) prime factors. This means that the average length of a valid list is only around $12 / 3.5 = 3.42$ (thus 3 to 4) numbers! From here we can conclude that, although there are valid sequences with 12 elements, *most* of the sequences will have only few. If we assume that in most cases the length of the list is no more than 6, our calculation for the number of reachable states would be $(6+2)^6 = 262,144$ – much closer to what we get empirically.

Approach 2 (map-based solution)

The contestants could exploit the low number of states, by using a map-based solution (instead of figuring out the perfect state). Since we don't care *where* in the list we have written each of the previous numbers, only which numbers we have written, we can instead use `map<vector<int>, int>` as a dynamic table. This approach is pretty simple and, as long as the vector is sorted, also relatively fast – it would get 80 points.

Approach 3 (smart solution)

We can do another observation (this time a hard one) to get a better state. We already saw that the number of reachable states is much lower than what our table covers. What if we make the table smarter (less sparse)?

We don't really care on which position each of the prime factors was used, only which pairs of prime factors cannot be used together anymore. For example, if we've already written 2 and 3, we know that we cannot use a number, which is divisible by 6. If we've written the numbers 6 and 35, we know that we cannot use the pairs (2, 5), (2, 7), (3, 5), and (3, 7) – thus the numbers 10, 14, 15, and 21 cannot be divisors of any new number.

How many pairs there are? Since (2, 3) and (3, 2) are the same, we'll only need information about $6 * 7 / 2 = 21$ pairs. And we can use a bitmask for that.

This brings the table to $2^{21} = 2,097,152$ states (slightly lower than the previous one). In fact, many of those are also not reachable. For example, if we know that the pairs (2, 3) and (5, 7) are forbidden, but (3, 5) is not, then we can conclude that (2, 5) should also be forbidden (because 3 and 5 were apparently used in the same number). Checking this empirically shows that only slightly over 3000 states were actually reached! It turns out that this solution is not only better in terms of memory efficiency, but also much faster as runtime as well. The number of states is so low, that even a map-based solution with this idea gets a full score.

I'd be interested in what is the best state the contestants come up with, since even this approach is very inefficient in terms of state.

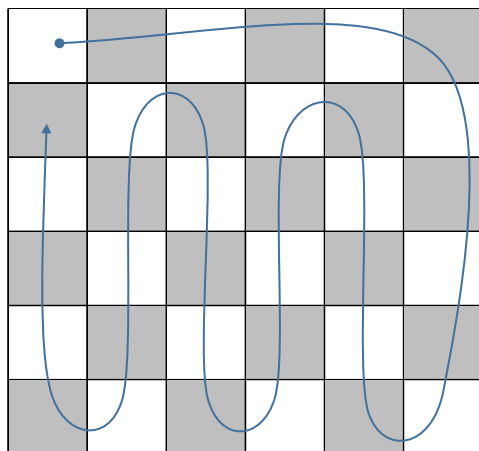
Naive (bruteforce) approaches

Of course, the contestants could have tried other simpler approaches – for example bruteforce. Depending on how smart the backtrack was written, they could have gotten quite a lot of points. This was done on purpose, since the smart solutions are on the hard side. The trivial bruteforce, for example, was supposed to get around 40 points. An optimized one could get around 60.

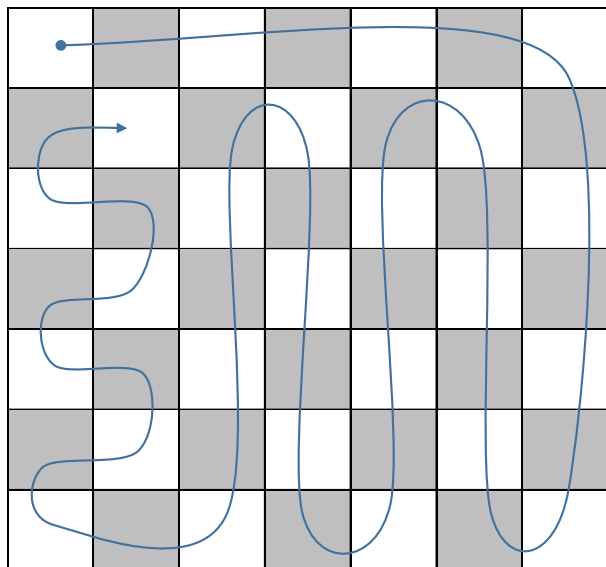
Analysis

The task concerns finding a Hamiltonian cycle in a graph, defined by some neighborhood on a rectangular network. Generally, it is an NP-complete problem and its solution needs exhausting algorithms. For small values of N this can be done here, too. For bigger values, some rule is to be found to realize the tour. The solution, if it exists, is even far not unique, which gives the chance for different rules to be noticed. A usual way is to split the given graph into smaller subgraphs, each with existing Hamiltonian path and after walking it, to provide a legal transition to the next subgraph. We use this method here as well.

The idea is to be able to construct a Hamiltonian path for a square with side of length 5, with given start and final positions. For to calculate quicker the first desired path using standard backtracking, we can often diminish the final demands to only the desired row or column, setting free the other dimension, as it, more often than not, does not matter. What counts is to get correctly to the next subgraph – a 5×5 square. If we can solve subtasks of this kind, we can divide the big board into smaller 5×5 squares, every one of which can be travelled without leaving its boundaries, finishing the path in a place, which assures a correct move to the next 5×5 square. If we denote the number of columns (and rows) of 5×5 squares with T , a Hamiltonian tour on it is easy to be seen. In the tours shown below every square is in fact a 5×5 square:



For even T



For odd T

It appears that we can use a small number of different Hamiltonian paths on a 5×5 square, less than 20. This makes it possible to remember them and reuse the found ones. They are identified by their start and end positions, as, once again, the final row or column usually do not matter.

Nothing remains but to take into consideration the necessity of leaving “steps” in the top left 5×5 corner: one or two unvisited little squares, leading to the top left little square. This should be the end of the path, transforming it into a cycle. One little square is enough for “stepping” when T is even, and, if T is odd, at least two little squares are needed. Of course, this is true for “natural” walks like the one above. As one can guess, the number of suitable paths on the board can be large.

Analysis

1 Introduction

The problem statement in short is:

We have a tree in which every vertex has a weight. We want to decompose the tree into vertical chains, such that every vertex belongs to exactly one chain and the sum of “values” of the chains is maximum. The value of a chain is defined as $W_{max} - W_{min}$, where W_{max} is the maximal weight of a vertex in the chain and a W_{min} is the minimal weight.

We want our solution to be $O(N \log N)$ or $O(N)$ in time, where N is the number of vertices in the tree.

2 Observations

To solve the problem we first need to make some observations. The most important one is that at least one of the following is true for every chain:

- The chain should start from the vertex with the largest weight and finish on the vertex with the smallest weight from this path.
- The chain should start from the vertex with the smallest weight and finish on the vertex with the largest weight from this path.

This is true, because if we have a path not satisfying one of the above conditions we can divide this path into several smaller ones without decreasing the initial value.

The second observation is that there exists an optimal decomposition into chains such that the weights of the vertices in each chain are either increasing or decreasing. This observation is again true because if there was a chain not satisfying this, we could have divided it into several smaller ones without decreasing its initial values.

3 Main part

Using the above observations we can solve the problem in linear time. The author’s solution uses only the first observation. Also there exists a solution which is easier to be implemented but it uses both observations. Both solutions are linear in time.

To get partial points one could have implemented a brute force for 10 points. To get 40 points one could implement a solution quadratic in time.

3.1 Solution for 40 points

We will solve the problem with dynamic programming.

Our state will be dp_u which is the maximal sum of values of chains in a decomposition of the subtree rooted at vertex u . We know that there is a chain starting at vertex u and finishing in a vertex in its subtree. Let's denote this vertex as v and the set of vertices which belong to the path from u to v as S . Also let's denote by C_i the sum of the dp values of the children of u ($C_i = \sum_{x \in \text{children}(i)} dp_x$). From here we get that for the chain starting at u and finishing at v , the value of dp_u will be $\sum_{x \in S} C_x - \sum_{x \in S \setminus \{u\}} dp_x + \max_{x \in S} w_x - \min_{x \in S} w_x$. And so we can find dp_u by checking this value for all vertices v in the subtree of u and then by getting the maximal value. This solution is $O(N^2)$. If implemented well it can get even 50 points.

3.2 Author's Solution

We will use dynamic programming to solve the problem. With each vertex we will associate 3 values:

- $dp_{u,0}$ – maximum sum of values to cover the subtree of u .
- $dp_{u,1}$ – maximum sum of values to cover the subtree of u , if we have an unfinished path, which includes the root and a fixed maximum (the value of the maximum has been added in advance).
- $dp_{u,2}$ – maximum sum of values to cover the subtree of u , if we have an unfinished path, which includes the root and a fixed minimum (the value of the minimum has been subtracted in advance).
- The answer to the problem will be $dp_{1,0}$.

We will use DFS, starting from the root (vertex number 1). After going through all of the children of a given vertex, we will calculate the three values for the vertex. Let us firstly calculate $dp_{u,1}$. The evaluation of $dp_{u,2}$ will use an analogical approach as the one for $dp_{u,1}$.

When we calculate $dp_{u,1}$, we have two possibilities:

- A new path starts from vertex u .
- A path, which until now finished in one of the children of u , continues through u .

In the first case, the value will be $W_u + \sum_{v \in \text{children}(u)} dp_{v,0}$.

In the second case, we need to choose a child of u , the path from which we want to continue. Let $S = \sum_{v \in \text{children}(u)} dp_{v,0}$. If the chosen child of u is p , the value of $dp_{u,1}$ will be $S - dp_{p,0} + dp_{p,1}$.

This way we can find the maximum value for $dp_{u,1}$ and $dp_{u,2}$. Now in order to calculate $dp_{u,0}$, we once again have two possibilities:

- To finish a path, where u is the vertex with maximum weight.
- To finish a path, where u is the vertex with minimum weight.

$$\text{So we get } dp_{u,0} = \max(dp_{u,1} - W_u, dp_{u,2} + W_u).$$

This solution has a complexity of $O(N)$ if it's implemented by the described above way.

3.3 Second Solution (Encho Mishinev)

We will once again use dynamic programming. However, we will use the second observation – that an optimal decomposition into paths exists, such that the values in each path are either only increasing or only decreasing. If we decompose the tree in such fashion, we can say, that the weight of a path is the sum of the absolute differences between each two consecutive vertices. For increasing or decreasing paths this definition is equivalent to $W_{\max} - W_{\min}$. With each vertex we will associate 2 values:

- $F_{v,0}$ – maximum sum of values for the covering of the subtree of u , if the path starting from u is with decreasing weights.
- $F_{v,1}$ – maximum sum of values for the covering of the subtree of u , if the path starting from u is with increasing weights.

The answer to the problem will then be $\max(F_{1,0}, F_{1,1})$

The calculation of $F_{v,0}$ and $F_{v,1}$ is analogical, so we will focus only on the calculation of $F_{v,0}$:

$$\text{Let } S = \sum_{x \in \text{children}(v)} \max(F_{x,0}, F_{x,1}).$$

Because a decreasing path starts from v , we have to choose to which child it continues. If this child is u ($W_u \leq W_v$), the value will be $S - \max(F_{u,0}, F_{u,1}) + (W_v - W_u) + F_{u,0}$.

We have to remember the case, where v is a path, consisting only of one vertex and thus the value is simply S .

This way with $O(1)$ we can evaluate the value for a fixed child, which allows us to calculate all values in the tree with a complexity of $O(N)$.

4 Additional information

You can try to solve a more difficult version of the problem. In it once again you need to maximize the sum of the paths, where the value of a path is once again the difference between the largest and smallest weight in the path, however we have one more condition – each path has to have $\text{length} \leq K$, where K is a given integer.

This version of the problem can be solved with a complexity of $O(N \log N)$, using a segment tree, or with a complexity of $O(N \log^2 N)$, using the technique, described here:

<http://codeforces.com/blog/entry/44351>.

Analysis

Exponential solution

For very small constraints we can simulate the games with exponential complexity – we simply try all possible ways the game could go. Since on every turn a player can have many different choices, this solution runs very slowly. It would work for about $N \leq 10$, depending on the implementation.

Polynomial solutions

The main observation in the problem is the following statement:

The optimal choice for the player whose turn it is is to take the largest number in the multiset.

This statement is obvious, since taking any other number does not obstruct the other player in any way and only reduces the player's own score.

In such case, we can just simulate all K games.

Solution for $O(N^2 \times K)$

On every turn, we have to find the largest number in the multiset. We can do this by simply iterating through the multiset linearly. Since its size can be at most $O(N)$, the complexity for the simulation of a single game is $O(N^2)$ and the total complexity of the solution is $O(N^2 \times K)$.

Solution for $O(N \times K \times \log(N))$

We can improve our previous solution by keeping the elements that are in the multiset in some structure that allows to quickly find the maximum number, as well as to remove the largest element and to add new elements. There are many structures that support these operations with logarithmic complexity per operation. The competitors can implement a structure of their own or use one of the STL structures – for example *priority_queue* or *set/map*. The best complexity that can be obtained using a structure of this kind is $O(N \times \log N)$ for the simulation of a single game and hence a total complexity of $O(N \times K \times \log(N))$.

Solution for $O(N \times K)$

In order to improve the efficiency of our simulation we have to make the following observation:

If in a certain moment the number that is being added to the multiset is the largest number in the multiset, then this number will be taken on the very next turn.

This statement follows directly from the optimal playing strategy. In such case, let us define:

$count[x]$ = the amount of times we have the number x in the multiset at a certain moment

Since all numbers are up to N , the size of $count[]$ is also N .

The optimal move at a certain moment is to take the largest x , such that $count[x] > 0$. Adding and removing numbers can be done in $O(1)$ time by simply increasing or decreasing the respective value in $count[]$ with 1. The algorithm to quickly simulate a single game is as follows:

We begin with a pointer $ptr = N$ and we want to have $count[x] = 0$ for every $x > ptr$ at any moment. To find the optimal choice at a certain moment we reduce the pointer until we reach $count[ptr] > 0$. When we are adding a new number A to the multiset, we have two options:

- 1) If $A > ptr$, then we remember that this number will be taken on the very next move. We don't have to update $count[]$ at all.
- 2) If $A \leq ptr$ then we can simply add the number and find the optimal choice using the pointer.

Since we reduce the pointer at most N times and process every addition in constant time, the total amortized complexity for the simulation of a single game is $O(N)$. Thus we get a total complexity of $O(N \times K)$.